

# OSN2526: Assignment 2: Page tables

Jenny Vermeltfoort

April 12, 2026

## 1 Introduction

This report describes an analysis of different virtual memory implementations. In Section 2 a comparative analysis is performed between a simple single level MMU and a RISC-V Sv48 MMU. Both MMU are simulated within the provided framework given for the assignment.

## 2 MMU Comparison

This section performs a comparative analysis of a simple single level MMU provided for the assignment and a RISC-V Sv48 MMU implementation.

Table 1 shows the results for a single process and Table 2 shows the results for multiple processes.

Table 1: Single process results of RISC-V Sv48 and Simple MMU Implementations for trace: gcc.txt.gz.

| Metric                          | RISC-V (Sv48) | Simple (Single-Level) | Difference |
|---------------------------------|---------------|-----------------------|------------|
| Page size                       | 4 KB          | 64 MB                 | -          |
| Page table allocated            | 36 KiB        | 64 MiB                | 67 MB      |
| Handled page faults             | 246           | 4                     | 2          |
| Max. # allocated physical pages | 237           | 5                     | 10         |

Table 2: Multiprocess results of RISC-V Sv48 vs. Simple MMU (gcc.txt.gz and ls.txt.gz)

| Metric                          | RISC-V (Sv48) | Simple (Single-Level) | Difference |
|---------------------------------|---------------|-----------------------|------------|
| Page size                       | 4 KB          | 64 MB                 | -          |
| Page table allocated            | 68 KiB        | 128 MiB               | 134 MB     |
| Handled page faults             | 479           | 7                     | 4          |
| Max. # allocated physical pages | 488           | 8                     | 20         |

These results nicely capture the impact of using a RADIX tree to manage the virtual memory translation. For a single level table each store of a physical memory is accompanied by a 64 MB page size allocation. When we have two physical addresses which are further apart than the size of the page the amount of bytes allocated easily duplicates. For the multilevel RISC-V Sv48, this issue is not as severe since the pages are only of size 4 KB. When a fragmented access occurs the relative allocation overhead is much

less. This can be seen in the results, the Sv48 implementation allocates 36 KiB for a single process compared to 64MiB for the single level implementation.

The multiprocess results show improved efficiency for the Sv48 implementation, each virtual table per process has only 4KB in size compared to the 64MB per process for the single level implementation. This does go at the cost of some performance. The implementation of the Sv48 is more complex, it needs to go through a couple of stages/levels to retrieve the physical address. Also, since the page size is lower it covers less physical address space, thus the amount of page faults increases.

### 3 TLB Comparison

A TLB (Translation Lookaside Buffer) is a method that caches the entries of the virtual page table to reduce memory latency. An entry within the TLB relates a tag, usually the virtual page number, to a physical address. In subsection 3.1 an analysis of the effectiveness of the size of the TLB is given; in the subsection 3.2 the performance of the TLB for multiprocesses using an ASID implementation is analysed.

#### 3.1 Size analysis

The table below depicts the results for a couple of different TLB configurations. The highest TLB size is chosen to be 64 entries since Intel's Nehalem microarchitecture similarly has a 64 entry 4 KiB page TLB. With this analysis and simulation its best to capture a scenario that best represents actual hardware.

In this analysis it is of interest where the tipping point of the size of the TLB lies in relation to the performance; performance in this case the hit rate for different applications. Each application might have a different memory access pattern, it is therefore of interest to relate the results of multiple applications to any conclusion derived from the results. As a disclaimer the simulation of this experiment does not implement the ASID selector, meaning for each context switch within the processor the TLB is flushed.

Results are depicted in Table 3.

Table 3: TLB Hit Rates by Size and Trace

| Trace File           | TLB=4  | TLB=8  | TLB=16 | TLB=32 | TLB=64 |
|----------------------|--------|--------|--------|--------|--------|
| cxx.txt.gz           | 91.07% | 93.80% | 98.86% | 99.66% | 99.87% |
| gcc.txt.gz           | 91.94% | 94.66% | 98.83% | 99.62% | 99.87% |
| ls.txt.gz            | 91.63% | 96.10% | 97.83% | 99.17% | 99.80% |
| make.txt.gz          | 89.67% | 94.23% | 97.52% | 99.06% | 99.84% |
| small-reclaim.txt.gz | 92.02% | 96.03% | 97.81% | 99.51% | 99.80% |
| small.txt.gz         | 92.02% | 96.03% | 97.81% | 99.52% | 99.81% |
| combined (all)       | 90.33% | 94.37% | 97.53% | 98.44% | 98.82% |

What is observable from these results is that all applications receive a performance benefit from a TLB size that is bigger than 32 entries. Where `small-reclaim.txt.gz` has the smallest improvement of 7.79 % going from a TLB of size 4 to size 64; similarly the biggest improvement of 10.17 % occurs with the trace `make.txt.gz`. See the boxplot in Figure 1 for a visual presentation of the average performance gain over all rows of Table 3.

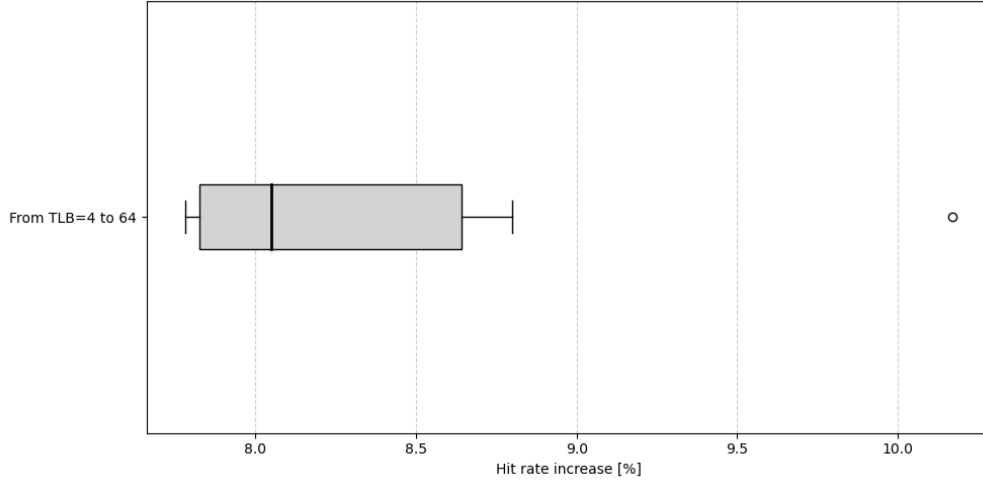


Figure 1: Boxplot of the average performance gain going from TLB=4 to TLB=64.

Another observation is going from a TLB of size 32 to a TLB of size 64 has a limited effect on the hit rate, depicted in Figure 2, with an average gain of roughly 0.4%.

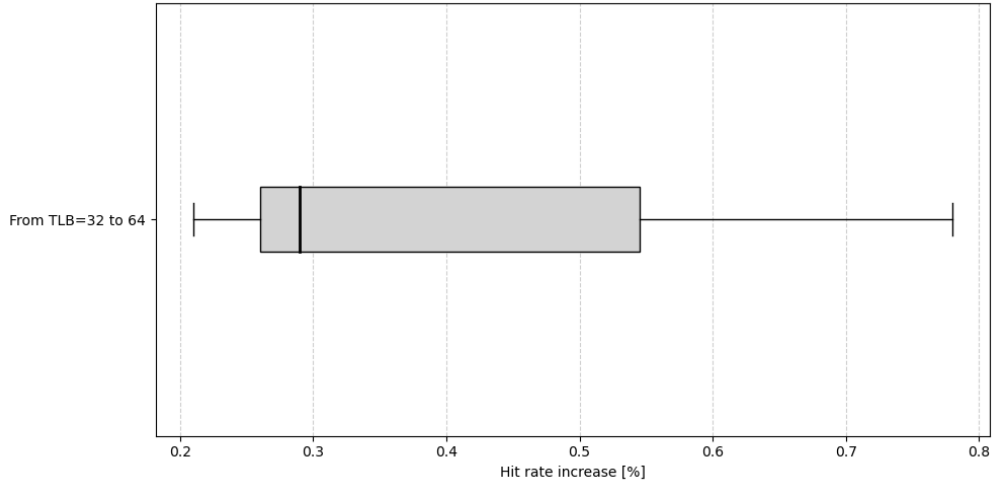


Figure 2: Boxplot of the average performance gain going from TLB=32 to TLB=64.

### 3.2 ASID analysis

The table below contains the results for the simulation on different configurations of traces with and without ASID. As shown in Figure 3 there is a 0.6 % average gain in hit rate.

Table 4: TLB hit rate comparison: Flushing vs. ASID

| Workload Configuration               | Flushing | ASID     |
|--------------------------------------|----------|----------|
| cxx.txt.gz + ls.txt.gz               | 98.6369% | 99.6405% |
| cxx.txt.gz + ls.txt.gz + make.txt.gz | 98.9326% | 99.5261% |
| All*                                 | 98.8153% | 99.1680% |

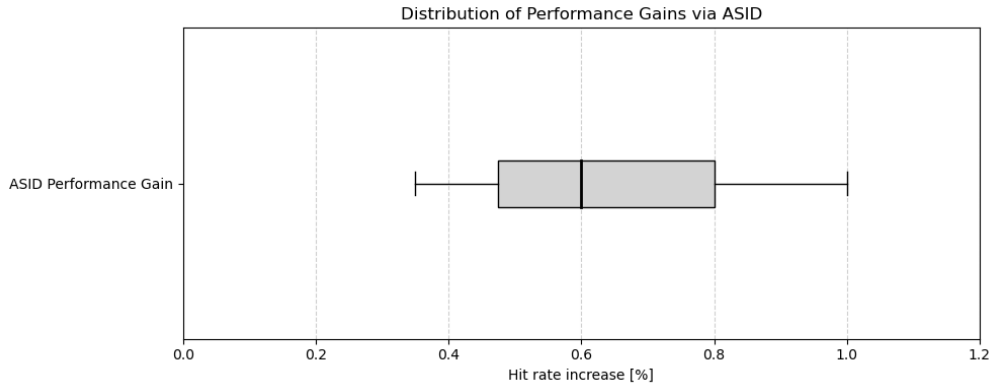


Figure 3: Boxplot of the average performance gain using ASID compared to Flushing.

\* Combination of: cxx, gcc, ls, make, small-reclaim, and small traces.